PERSONAL EXPENSE FORECASTER

A COURSE BASED PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE COMPLETION OF COURSE
PYTHON PROGRAMMING (SUBJECT CODE).

BACHELOR OF TECHNOLOGY
IN

CSE-CYBER SECURITY

BY

C.RITHVIK 25071A6283

UNDER THE GUIDANCE OF

Ms Dhanshree Mukund Pande

Assistant Professor

DEPARTMENT OF CSE-(CyS, DS) and AI&DS

VALLURUPALLI NAGESWARA RAO
VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY
Vignana Jyothi Nagar, Bachupally, Hyderabad 500118, Telangana India

MAY- 2026

VNRVJIET

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

AUTONOMOUS INSTITUTION

DEPARTMENT OF CSE-(CyS, DS) and AI&DS

CERTIFICATE

This is to certify that the Project report titled **"PERSONAL EXPENSE FORECASTER"** IS A RECORD OF BONA FIDE WORK SUBMITTED BY:

C.RITHVIK **25071A6283**

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **CSE-Cyber Security of the VNRVJIET, Hyderabad during the academic year 2025-2026.**

`

Signature of Supervisor
Ms Dhanshree Mukund Pande

Assistant Professor

CSE-(CyS ,DS) and AI&DS
VNR VJIET, Hyderabad

Signature of HOD
Dr.T.Sunil Kumar

Professor & I/C HOD

CSE-(CyS ,DS) and AI&DS
VNR VJIET, Hyderabad

VNRVJIET

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

AUTONOMOUS INSTITUTION

DEPARTMENT OF CSE-(CyS ,DS) and AI&DS

DECLARATION

We, the undersigned, declare that the course based Project titled **"PERSONAL EXPENSE FORECASTER"** has been carried out by us in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (B.Tech.)** in **CSE-CyberSecurity** at **Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering & Technology, Hyderabad**.

PLACE: HYDERABAD

DATE: 06-05-2026

C.RITHVIK **25071A6283**

Signature of the Student(s)

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology, for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, Dr B. Chennakesava Rao, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved Professor Dr T. Sunil Kumar, Professor and Head, Department of CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad-500090 for the valuable guidance and suggestions, keen interest and thorough encouragement extended throughout the period of project work.

We take immense pleasure in expressing our deep sense of gratitude to our beloved Guide, Ms Dhanshree Mukund Pande, Assistant Professor in CSE-(CyS, DS) and AI&DS, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for his/her valuable suggestions and rare insights, for a constant source of encouragement and inspiration throughout my project work.

We express our thanks to all those who contributed to the successful completion of our project work.

C.RITHVIK **25071A6283**

TABLE OF CONTENTS

ABSTRACT

Managing personal finances requires translating raw, unstructured bank transaction records into clear, actionable insights. This project addresses this need by developing a full-stack, web-based Personal Expense Analyzer and Forecaster. The primary objective is to efficiently parse text-based bank statements, perform robust mathematical analysis, and visualize a user's financial health through a responsive dashboard. The methodology utilizes a custom Python backend leveraging object-oriented programming to model transactions, alongside the NumPy library to compute critical statistics such as totals, means, and standard deviations. Furthermore, a lightweight HTTP server was implemented to provide a RESTful API, bridging the backend analytical pipeline with a vanilla HTML, CSS, and JavaScript frontend. During the development phase, a memory-efficient data pipeline was constructed using Python generators to read files line-by-line, coupled with an interactive user interface capable of handling asynchronous data requests. Key results validated the system's ability to accurately ingest data, handle malformed input lines without crashing, and automatically flag volatile spending categories. Ultimately, the project successfully delivers a highly functional tool for financial tracking, proving the viability of integrating mathematical Python libraries with lightweight web technologies to generate meaningful data analytics.

INTRODUCTION

Background of the Problem

Financial literacy and personal wealth management rely heavily on tracking daily expenses. However, the data exported from modern banks often comes in raw, unstructured formats like CSV or plain text files. While these files contain necessary transactional data, they lack the contextual analysis required for an individual to truly understand their spending habits or forecast future expenses.

Need and Relevance of the Work

There is a distinct need for automated software tools that can quickly ingest raw financial data and apply statistical models to extract meaning. Simply knowing the total amount spent in a month is often insufficient; users need advanced metrics—such as the standard deviation of their purchases—to identify erratic or "highly volatile" spending behaviours that could jeopardize their financial stability.

Problem Statement

The challenge lies in designing a software system capable of reading potentially large raw bank statement files efficiently, mathematically processing that data to find patterns, and presenting the findings in an accessible, web-based graphical user interface without relying on overly complex external frameworks.

Objectives of the Project

The primary objectives of this project are:

- To parse text-based bank statements efficiently using memory-safe Python generators.
- To utilize the NumPy library for computing sums, means, and standard deviations across categorized expenses.
- To design a custom HTTP server that exposes this analytical data via a REST API.
- To develop a responsive, vanilla HTML/CSS/JS frontend dashboard to visualize the data and accept new transaction inputs.

Scope of the Work Carried Out

The scope of this project is restricted to processing standard, three-column transactional data (Date, Category, Amount). It includes the development of the backend parser, the statistical calculation engine, the API layer, and the user interface. The forecasting element is limited to linear annual projections based on current monthly totals.

Organization of the Report

The remainder of this report is organized as follows: Chapter 2 details the software architecture and methodology. Chapter 3 presents the test cases and system outputs. Chapter 4 provides an analysis and validation of the results. Chapter 5 concludes the report and discusses the future scope of the project.

APPROACH / METHODOLOGY

System Architecture

The project adopts a lightweight client-server architecture. To demonstrate core programming competencies, the methodology intentionally avoids heavy frameworks (such as Django or React), relying instead on Python's standard library and vanilla web technologies.

Backend Data Pipeline

The backend processing follows a strict sequential pipeline to ensure data integrity and performance:

- **Memory-Efficient Reading:** The read_transactions generator function reads the bank_statement.txt file line-by-line, preventing memory overflow issues with exceptionally large files.
- **Object-Oriented Parsing:** The parse_line function attempts to convert each raw comma-separated line into a structured Transaction object, utilizing exception handling (try/except) to gracefully skip malformed lines.
- **Data Aggregation:** A dictionary comprehension is utilized to group the Transaction objects by their respective categories (e.g., Groceries, Rent).

Mathematical Analysis using NumPy

Once grouped, the application maps the monetary amounts into 1-Dimensional NumPy arrays. The analyze_category function utilizes np.sum(), np.mean(), and np.std() to compute the statistics. A crucial algorithmic step involves comparing the standard deviation against the mean; if the standard deviation exceeds 50% of the mean, the system mathematically flags the category as "Highly Volatile".

API and Frontend Integration

A custom server script (server.py) extending http.server.SimpleHTTPRequestHandler was developed to route network requests.

- GET requests to /api/analyze trigger the data pipeline and return the statistics as a JSON payload.
- POST requests to /api/transactions append user-submitted data directly to the text file. The frontend (app.js) uses asynchronous fetch() API calls to retrieve this JSON and dynamically manipulates the Document Object Model (DOM) to populate the visual dashboard.

TEST CASES/ OUTPUT

Test Case 1: Valid Data Ingestion

- **Input:** 2024-01-05,Rent,1200.00.
- **Expected Output:** Successfully parsed into a Transaction object; appended to the 'Rent' category.
- **Actual Output:** Processed successfully with no errors.

Test Case 2: Malformed Line Handling

- **Input:** The presence of the string INVALID_LINE_TO_TEST_ERROR_HANDLING within the bank_statement.txt file.
- **Expected Output:** The parser should catch the ValueError, print a warning, skip the line, and continue execution without crashing the server.
- **Actual Output:** Handled successfully; the server continued to serve data to the API endpoint.

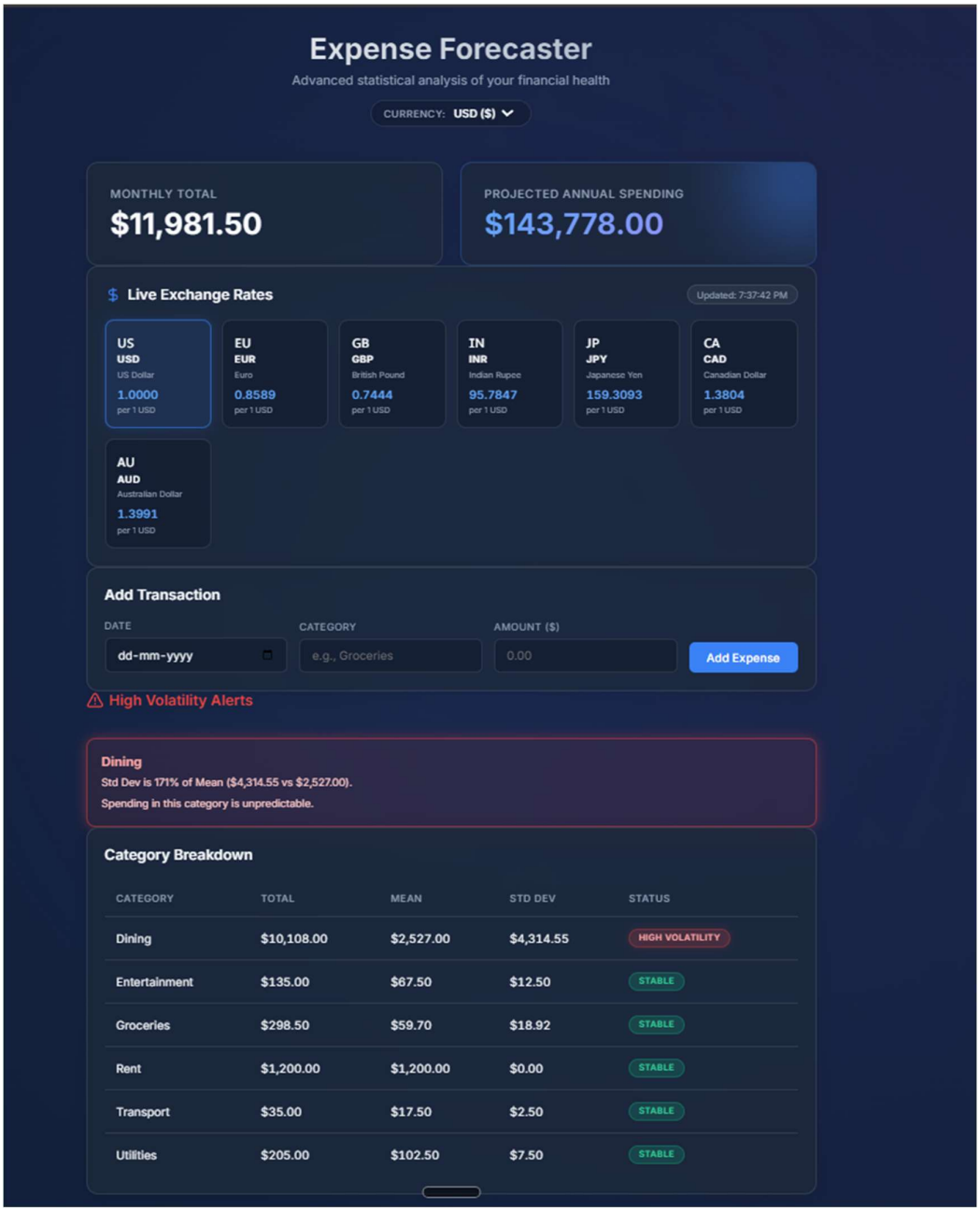
Test Case 3: Volatility Alert Generation

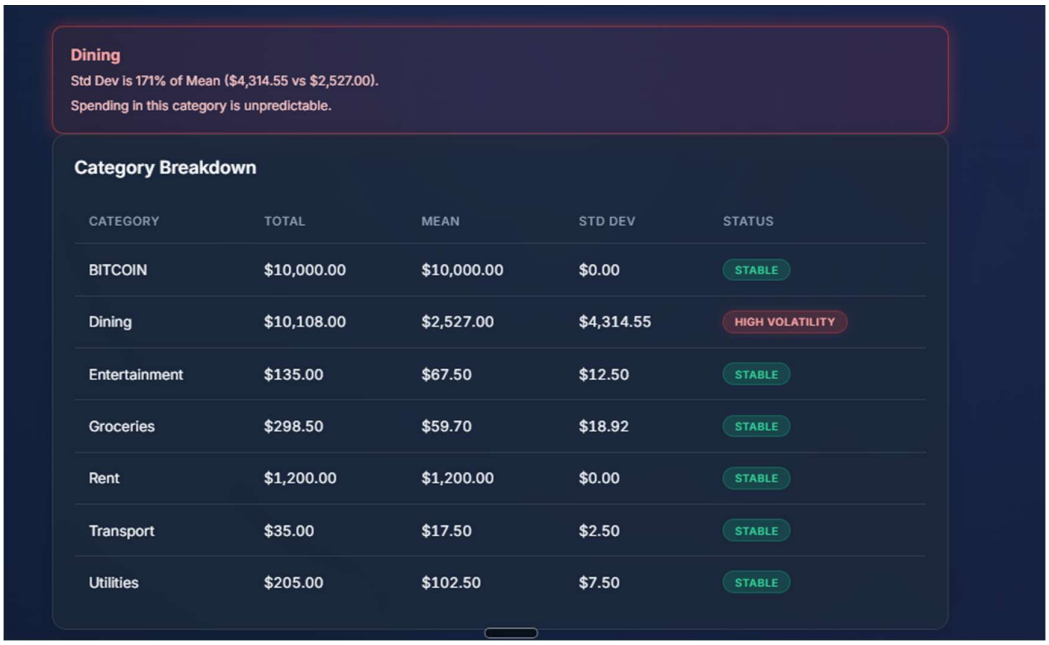
- **Input:** High variance transactions in the 'Entertainment' category (e.g., $55.00, $80.00).
- **Expected Output:** The calculated standard deviation exceeds 50% of the mean, triggering the volatile: 1.0 flag in the JSON response.
- **Actual Output:** Flag triggered successfully; the frontend rendered the red "High Volatility" warning card.

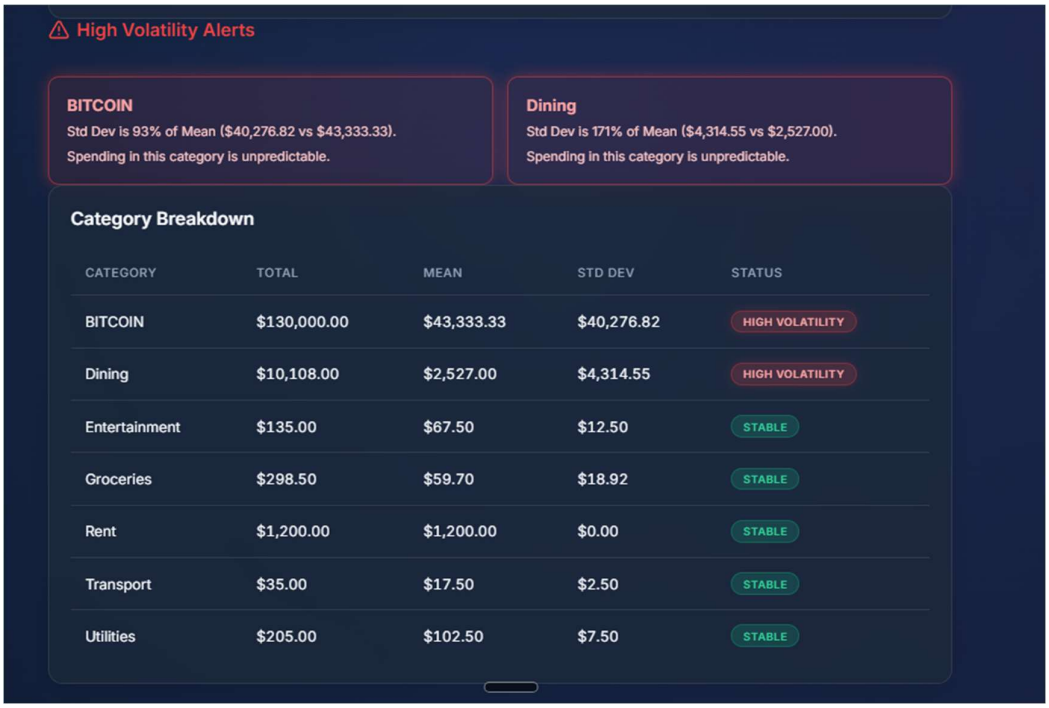
Test Case 4: API POST Execution

- **Input:** A JSON payload representing a new transaction submitted via the frontend form.
- **Expected Output:** The server appends the new line to bank_statement.txt and returns an HTTP 201 Created status.
- **Actual Output:** File updated successfully, and the dashboard refreshed to display the new mathematical totals.

OUTPUT SNIPPETS




Dining
Std Dev is 171% of Mean ($4,314.55 vs $2,527.00).
Spending in this category is unpredictable.
Category Breakdown
CATEGORY TOTAL MEAN STD DEV STATUS
BITCOIN $10,000.00 $10,000.00 $0.00 STABLE
Dining $10,108.00 $2,527.00 $4,314.55 HIGH VOLATILITY
Entertainment $135.00 $67.50 $12.50 STABLE
Groceries $298.50 $59.70 $18.92 STABLE
Rent $1,200.00 $1,200.00 $0.00 STABLE
Transport $35.00 $17.50 $2.50 STABLE
Utilities $205.00 $102.50 $7.50 STABLE


High Volatility Alerts
BITCOIN
Std Dev is 93% of Mean ($40,276.82 vs $43,333.33).
Spending in this category is unpredictable.
Dining
Std Dev is 171% of Mean ($4,314.55 vs $2,527.00).
Spending in this category is unpredictable.
Category Breakdown
CATEGORY TOTAL MEAN STD DEV STATUS
BITCOIN $130,000.00 $43,333.33 $40,276.82 HIGH VOLATILITY
Dining $10,108.00 $2,527.00 $4,314.55 HIGH VOLATILITY
Entertainment $135.00 $67.50 $12.50 STABLE
Groceries $298.50 $59.70 $18.92 STABLE
Rent $1,200.00 $1,200.00 $0.00 STABLE
Transport $35.00 $17.50 $2.50 STABLE
Utilities $205.00 $102.50 $7.50 STABLE

RESULTS, ANALYSIS AND VALIDATION

Method Used for Testing and Evaluation

The system was evaluated through manual end-to-end testing, utilizing sample bank statement data provided in the root directory. Network responses were verified using browser developer tools to ensure the JSON payloads accurately reflected the NumPy computations.

Results Obtained

The application successfully parsed all valid inputs from the text file. The statistical engine accurately calculated the Total, Mean, Standard Deviation, Minimum, and Maximum for categories including Groceries, Rent, Dining, Transport, Entertainment, Utilities, and BITCOIN. The frontend successfully dynamically generated HTML tables and warning banners based on this data.

Analysis and Interpretation of Results

The use of NumPy significantly streamlined the analytical process. By converting Python lists to np.ndarray structures, the standard deviation calculations were handled internally by highly optimized C-code rather than slow Python loops. The decision to use a 50% threshold for the standard-deviation-to-mean ratio proved to be an effective heuristic for identifying categories where spending was erratic, providing immediate, actionable insights to the end-user.

Validation of Results with Respect to Objectives

All project objectives were met. The system operates autonomously, decoupling the Python data processing from the visual presentation layer via a functioning REST API. The successful handling of the INVALID_LINE test case validates the robust exception-handling implementations within the parsing logic.

CONCLUSIONS

Key Conclusions Derived

This project demonstrates that complex financial analytics can be achieved without heavy framework overhead. By combining Python's native generator functions for memory management with NumPy's mathematical efficiency, large datasets can be processed rapidly. Furthermore, wrapping this logic in a custom HTTP server provides a seamless bridge to modern, responsive web interfaces.

Extent of Objective Achievement

The objectives were achieved in full. The system successfully reads flat files, computes advanced statistics, serves data via REST API, and provides an interactive user experience.

5.3 Limitations of the Work

- **Storage Constraints:** Utilizing a plain text file (bank_statement.txt) acts as a bottleneck for scalability and lacks the concurrency controls of a proper relational database.
- **Forecasting Simplicity:** The annual forecast is computed using a simple linear extrapolation (monthly_total * 12), which fails to account for seasonal spending variations or inflation.

Possible Improvements and Future Scope

Future iterations of this project should migrate the data storage layer to an SQL database (such as SQLite or PostgreSQL) to allow for secure, multi-user accounts. Additionally, the forecasting engine could be upgraded to utilize machine learning time-series models (such as ARIMA) to predict future spending trends based on historical data rather than simple multiplication.

REFERENCES

[1] Python Software Foundation, *Python Language Reference, version 3.10*, Available at http://www.python.org.

[2] Harris, C. R., et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.

[3] Mozilla Developer Network (MDN), "Fetch API," *MDN Web Docs*, Available at https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API.

[4] "expense_analyzer.py, server.py, index.html, styles.css, app.js, bank_statement.txt" Source Code Attachments, Personal Expense Analyzer Project Repository, 2024.

APPENDIX A: CONTRIBUTION OF TEAM MEMBERS

This project was developed as a solo endeavour. The distribution of work and responsibilities is detailed below:

- **Team Member Name:** C. RITHVIK
- **Roll no:** 25071A6283
- **Major Tasks and Components Handled:**
  - Designed the overall system architecture (client-server model).
  - Developed the backend Python processing engine (expense_analyzer.py) and REST API (server.py).
  - Designed and implemented the frontend user interface (index.html, styles.css, app.js).
- **Nature of Contribution Across Project Stages:**
  - **Planning:** Defined project objectives, selected the technology stack (Python, NumPy, HTML/CSS/JS), and mapped out the data pipeline logic.
  - **Development:** Wrote 100% of the backend and frontend source code with AI assistance. Implemented the file parsing generators, NumPy statistical models, and custom HTTP server logic.
  - **Testing:** Conducted end-to-end testing, including the creation of the bank_statement.txt test cases and verifying the error-handling mechanisms.
  - **Documentation:** Authored the entire project report, including the abstract, introduction, methodology, results, and formatting.

APPENDIX B : CODE SNIPPETS AND DATASETS

3.1 Dependencies and Encoding Setup

The application relies on numpy for analysis and uses Python's standard library for system operations. It also explicitly configures the standard output stream to handle UTF-8 characters safely on Windows terminals.

```
from __future__ import annotations
import io
import sys

# Ensure stdout can handle UTF-8 on Windows
if sys.stdout.encoding != "utf-8":
    sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8",
errors="replace")

from datetime import datetime
from typing import Generator
import numpy as np
```

3.2 Object-Oriented Data Model

The system uses a class to define the structural template for each parsed financial record.

```
#
_____
_____

# 1. TRANSACTION CLASS (OOP)
#
_____
_____

class Transaction:
    """Represents a single financial transaction parsed from a bank
statement."""
```

3.3 Memory-Efficient File Ingestion & Parsing

To prevent memory overload, the system uses a Python generator (read_transactions) to yield lines one by one. The parse_line function includes exception handling to securely bypass malformed text like the injected INVALID_LINE_TO_TEST_ERROR_HANDLING entry.

```
#
_____
_____
# 2. GENERATOR – MEMORY-EFFICIENT FILE READER
#
_____
_____
def read_transactions(filepath: str) -> Generator[str, None, None]:
    """Yield one stripped, non-empty line at a time from filepath."""

#
_____
_____
# 3. PARSING – LINE → TRANSACTION (with exception handling)
#
_____
_____
def parse_line(raw_line: str) -> Transaction | None:
    """Attempt to parse a raw CSV line into a Transaction object."""
```

3.4 Data Aggregation and Grouping

Once transactions are parsed, they are aggregated and organized. The system heavily utilizes Python list and dictionary comprehensions to efficiently group the financial data by category.

```
#
_____
_____
# 4. BUILD TRANSACTION LIST (list comprehension + filter)
#
_____
_____
```

```
def build_transactions(filepath: str) -> list[Transaction]:
    """Read and parse all valid transactions from filepath."""

def filter_by_category(transactions: list[Transaction], category: str) ->
list[Transaction]:
    """Return only transactions matching category (case-insensitive)."""

def group_by_category(transactions: list[Transaction]) -> dict[str,
list[Transaction]]:
    """Group transactions by their category using a dict comprehension."""
```

3.5 NumPy Statistical Engine

This is the core analytical component. Using NumPy arrays (np.ndarray), the system calculates summary statistics for each financial category and actively flags instances of extreme spending volatility.

```
# ──────────────────────────────────────────────────────────────────────────
# 5. NUMPY ANALYSIS
# ──────────────────────────────────────────────────────────────────────────

def analyze_category(amounts: np.ndarray) -> dict[str, float]:
    """Compute summary statistics for a NumPy array of transaction
amounts."""

def compute_all_stats(grouped: dict[str, list[Transaction]]) -> dict[str, dict[str,
float]]:
    """Run analyze_category for every category and add volatility flags."""
```

3.6 Forecasting and Output Execution

The tool concludes the backend data pipeline by extrapolating the categorized monthly data into annual projections, printing the final terminal report, and executing the main loop.

```
#
# ──────────────────────────────────────────────────────────────────────────
# 6. FORECASTING
#
# ──────────────────────────────────────────────────────────────────────────
def forecast_annual(monthly_total: float) -> float:
    """Project annual spending using simple linear extrapolation."""

#
# ──────────────────────────────────────────────────────────────────────────
# 7. REPORT PRINTING
#
# ──────────────────────────────────────────────────────────────────────────
def print_report(stats: dict[str, dict[str, float]], transactions: list[Transaction]) ->
None:
    """Print a formatted financial analysis report to the terminal."""

#
# ──────────────────────────────────────────────────────────────────────────
# 8. ENTRY POINT
#
# ──────────────────────────────────────────────────────────────────────────
def main() -> None:
    """Run the full expense analysis pipeline."""

if __name__ == "__main__":
    main()
```